



AP

Assignment 2

Anders Hoeck, Marcus Frehr Friis-Hansen

September 2026

1 Introduction

The goal of this assignment was to extend the interpreter from the exercises with a notion of state, and implementing support for printing values, support for key-value store and finally implementing a type checker. Implementation of this assignment was pretty straight forward, but some parts required extra interpretation, for this purpose we used LLM's for better understanding. LLM's has also been used for reasoning about code, debugging, syntax errors etc. We also used it to reason about the numbered questions. Especially for the final question we where unsure about how to argue about the Monad and Monoid laws. The LLM's used were ChatGPT and Claude.

We consider our solution to be functional. Functionality is covered by test for Print, KvPut, KvGet and CheckExp. The tests ensures the code is functional, however we recognize that we might not have covered all edge cases. All test cases currently pass, so we consider the quality of the code to be good.

2 Numbered Questions

2.1

What is the asymptotic complexity of your implementation of Print? Can this be improved?

The asymptotic complexity of our print is $O(n^2)$. The *evalPrint* function uses concatenation, to append the next element, to the list of strings. In Haskell, appending strings, or a list of strings, traverses each element from left to right. Therefore the larger the list of strings, the higher the cost of the concatenation operation `++`. The state of length `n` cost $O(n)$ to append. Therefore:

$$\sum_1^n n = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} = O(n^2)$$

Please note that strings essentially are list of chars, so we are working with a list of char lists.

The complexity can be improved to $O(n)$ by prepending the strings to the lists instead of appending. The cost each time we prepend a string becomes $O(1)$. And then finally at the end we can reverse the list once, to get the list in the correct ordering.

2.2

In your implementation, if a program keeps updating the same key with KvPut, does the memory usage of the program remain constant or does it increase

Our implementation of `kvPut` uses `evalKvPut` to update the keys and values. We initially look at just the key of each pair, and keep only the pairs where the key-value is not equal to `k`. We then add the new pair to the front of the list.

```
evalKvPut :: Val -> Val -> EvalM ()
evalKvPut k v = EvalM $ \_env (prints, store) ->
  ((prints, (k, v) : filter ((/=k).fst) store), Right ())
```

Therefore, if a program keeps updating the same key with `kvPut`, the memory usage remains constant, as any other pair, with the same key-value is removed before the update.

2.3

In `TryCatch e1 e2`, if `e1` fails after performing some effects, are those effects visible in `e2`, and which of your tests demonstrate this? If you wanted different behavior (either making the effects visible or invisible), what would you need to change in your implementation? Show exactly what the new code would look like

In the case where `e1` fails after performing some effect, those effects are still visible in `e2`. An effect is anything that changes the state, either using `Print` or `KvPut`. Visible in the `catch` function below, in the case where `e1` fails, it results in `Left`, it then runs `e2` with the new updated `state`'

```
eval (TryCatch e1 e2) =  
    eval e1 'catch' eval e2
```

```
catch :: EvalM a -> EvalM a -> EvalM a  
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->  
    case m1 env state of  
        (state', Left _) -> m2 env state'  
        (state', Right x) -> (state', Right x)
```

We use the test shown below to verify that the state is updated using `print` from `e1` after it fails, and is visible in the output after `e2` succeeds. Our implementation passes this test.

```
testCase "TryCatch keeps prints" $  
    eval' (TryCatch (Add (Print "a" (CstInt 1)) (Div (CstInt 1) (CstInt 0)))  
                (CstInt 2))  
    @?= ([ "a: 1" ], Right (ValInt 2))
```

If we wanted to make this behavior invisible, we would have to pass the original state to `e2`, instead of the updated state'. The new code would look like this:

```
catch :: EvalM a -> EvalM a -> EvalM a  
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->  
    case m1 env state of  
        (_, Left _) -> m2 env state  
        (state', Right x) -> (state', Right x)
```

2.4

Would it be possible to define type class instances

```
instance Monad m => Semigroup (m ()) where
    ...
instance Monad m => Monoid (m ()) where
    ...
```

and what might the definitions for *mempty* and *pure* look like? If so, argue based on the *monad* laws that your instances satisfy the *monoid* laws.

Yes we can define `<>` and `mempty`, given that *m* is a monad, like shown below

```
instance Monad m => Semigroup (m ()) where
  — (<>) :: m () -> m () -> m ()    semigroup requirement
  a <> b = a >>= \_ -> b

instance Monad m => Monoid (m ()) where
  — mempty :: m ()                    monoid requirement
  mempty = pure ()
```

The semigroup instance combines two computations of the type *m ()*, meaning the monad *m* returns nothing of significance. It initially runs the first computation *a*, then since we do not care about the result, we drop it using a lambda function, and run the second computation *b*. We chain the two computations together using `>>=`, which takes a computation and a function.

The monoid instance requires that we supply a neutral computation *mempty*, that does nothing when combined with another computation. Since we know that *m* is a monad, we can use *pure* for this purpose, to return `()`.

Since *m* is a monad, which we know satisfies the laws of left identity, right identity, and associativity.¹ It then follows that the instances above also satisfies the monoid laws of identity, and associativity.²

¹https://wiki.haskell.org/index.php?title=Monad_laws

²https://diku-dk.github.io/ap-notes/chapter_1.html